

Stiffness in the Robertson Chemical Kinetics System: What Explicit Methods Pay, What Implicit Methods Buy

Project 1 (ODE IVP) — Group O

Zihan Xu (2360751) — Project Manager, Visualisation & Report
Beixue Liang (2359957) — Mathematical Theory, Testing & Validation
Yumo Li (2253567) — Algorithm Implementation
Numerical Analysis of ODEs and PDEs, Project 1 (Weeks 1–4)

September 30, 2026

Abstract

We study numerical solution of the Robertson stiff chemical kinetics system (Classic Problem Pack, direction ③) on $0 \leq t \leq 40$. We build a small solver library from scratch (explicit Euler, classical RK4, implicit Euler with Newton’s iteration, and an adaptive step-doubling RK4), construct our own high-accuracy reference oracle, and answer one mathematical question: *how does stiffness — quantified by the frozen-Jacobian eigenvalues of the system — translate into a practical step-size restriction for explicit methods, and what does an implicit method pay for the same accuracy?* Our measurements show (i) the empirical explicit-Euler stability threshold $h \approx 0.0006$ agrees with the frozen-Jacobian prediction 0.00059 to about 2%; (ii) at a *matched* error target on the full interval the implicit method is the cheapest by one to three orders of magnitude in right-hand-side evaluations, because the explicit methods are pinned at their stability limits and therefore cannot trade accuracy for speed; and (iii) adaptivity removes the accuracy constraint but *not* the stability constraint: the global error of adaptive RK4 plateaus near 10^{-6} as the tolerance is tightened, because the step size is pinned at the RK4 stability limit. Stiffness is a stability story, and no controller can rewrite it.

Contents

1	Background and problem choice	2
2	Model	2
3	Methods	3
3.1	Explicit Euler (order 1)	3
3.2	Classical RK4 (order 4)	3
3.3	Implicit (backward) Euler with Newton’s iteration (order 1)	3
3.4	Adaptive step-doubling RK4	3
4	Stability analysis	4
4.1	Absolute stability regions	4
4.2	Frozen-Jacobian eigenvalues along the trajectory	4
4.3	Stiffness ratio	4
5	Validation: our own oracle	5

6	Numerical experiments	6
6.1	Observed convergence orders	6
6.2	Explicit failure and the empirical stability threshold	6
6.3	Cost vs. accuracy at matched error	6
6.4	Adaptive step-doubling RK4	8
6.5	Robertson diagnostics as separate signals	9
7	Conclusions	9
A	Individual Contribution Statements (ICS)	10
B	AI Transparency Log	12

1 Background and problem choice

Chemical reaction systems are the canonical example of *stiff* initial value problems: fast intermediate species and slow product species coexist in one state vector, so the Jacobian of the right-hand side has eigenvalues of very different magnitude. The Robertson system [1] is the standard benchmark for this phenomenon: its rate constants span eight orders of magnitude ($k_1 = 0.04$ vs. $k_2 = 3 \times 10^7$), yet along the actual trajectory the *realised* eigenvalue spread grows over time to about $S(40) \approx 1.584 \times 10^5$.

We chose direction ③ for three reasons. **(1) A sharp, testable question.** The Problem Pack gives a concrete explicit-stability estimate ($h < 2/|\lambda|_{\max} \approx 5.9 \times 10^{-4}$ at $t = 40$) that we can compare against our own measured threshold — a clean falsification experiment, not just a demo. **(2) A demanding validation story.** Explicit methods genuinely fail here (they oscillate, go negative, and blow up), which forces us to build serious diagnostics: an independent oracle, invariant-plane tracking, componentwise positivity, and a convergence-order check on the same baseline. **(3) A lesson with a twist.** As we show in Section 6.3, the “obvious” conclusion — implicit methods win on stiff problems — is only true when the accuracy target is loose; the quantitative crossover is something we measured rather than assumed.

All numbers in this report are produced by `code/run_all.py`; every figure regenerates from scratch in one command, and every quoted value is stored in `results/results.json`.

2 Model

With $\mathbf{y} = [y_1, y_2, y_3]^T$ (concentrations of species A, B, C) the Robertson system is

$$\begin{aligned} y_1' &= -0.04 y_1 + 10^4 y_2 y_3, \\ y_2' &= 0.04 y_1 - 10^4 y_2 y_3 - 3 \times 10^7 y_2^2, \\ y_3' &= 3 \times 10^7 y_2^2, \end{aligned} \tag{1}$$

with $\mathbf{y}(0) = [1, 0, 0]^T$ on $0 \leq t \leq 40$ (the reproducible protocol of the Problem Pack: output at $t = 0$ and 250 logarithmically spaced times from 10^{-8} to 40, identical for every method).

Conservation law. The right-hand side satisfies $(1, 1, 1)^T f(\mathbf{y}) = 0$, hence $y_1 + y_2 + y_3 = 1$ is an invariant plane. The full 3×3 Jacobian

$$J(\mathbf{y}) = \begin{bmatrix} -0.04 & 10^4 y_3 & 10^4 y_2 \\ 0.04 & -10^4 y_3 - 6 \times 10^7 y_2 & -10^4 y_2 \\ 0 & 6 \times 10^7 y_2 & 0 \end{bmatrix} \tag{2}$$

therefore always has one structurally zero eigenvalue. A stiffness ratio that includes it would be undefined, so following the Problem Pack we define $S(t) = \max |\operatorname{Re} \lambda| / \min |\operatorname{Re} \lambda|$ over the two

nonzero eigenvalues (zero classified by the threshold $|\operatorname{Re} \lambda| < 10^{-8}$), and begin the plot at the first positive output time.

Scales. y_2 (the radical B) is a fast transient: it rises to $\approx 3.6 \times 10^{-5}$ within $t \sim 10^{-4}$ and then relaxes to 9.19×10^{-6} by $t = 40$. This five-order-of-magnitude separation between the state components is exactly why a purely absolute error test is dangerous for the adaptive controller (Section 6.4).

3 Methods

All methods are implemented from scratch in `code/solvers.py` with a swappable $f(t, \mathbf{y})$ and (where needed) analytic Jacobian. Cost counters (right-hand-side evaluations, Jacobian evaluations, Newton iterations, accepted/rejected steps) are accumulated inside the solvers so the cost accounting in Section 6.3 is measured, not estimated.

3.1 Explicit Euler (order 1)

$y_{n+1} = y_n + hf(t_n, y_n)$. Local truncation error $\tau_n = \frac{h^2}{2}y''(t_n) + O(h^3)$, global order 1.

3.2 Classical RK4 (order 4)

$$\begin{aligned} k_1 &= f(t_n, y_n), & k_2 &= f(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1), \\ k_3 &= f(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2), & k_4 &= f(t_n + h, y_n + hk_3), \\ y_{n+1} &= y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \end{aligned}$$

Local truncation error $O(h^5)$, global order 4 (we verify the observed order numerically rather than relying on theory).

3.3 Implicit (backward) Euler with Newton's iteration (order 1)

The stage equation

$$g(v) = v - y_n - hf(t_{n+1}, v) = 0 \quad (3)$$

is solved by Newton's method,

$$(I - hJ(t_{n+1}, v^{(k)})) \delta^{(k)} = -g(v^{(k)}), \quad v^{(k+1)} = v^{(k)} + \delta^{(k)}, \quad (4)$$

with the analytic Jacobian J , initial guess $v^{(0)} = y_n$, relative stopping tolerance $\|\delta\|_\infty \leq 10^{-10} \max(1, \|v\|_\infty)$ and at most 20 iterations. The Newton tolerance is chosen so that the algebraic error is negligible against the time-discretisation error (a tolerance sweep confirmed that tightening it further does not change the reported digits). The local truncation error is $-\frac{h^2}{2}y''(t_n) + O(h^3)$ — the mirror image of explicit Euler — so both methods carry the same order-1 error constant in magnitude; they differ in stability, not in accuracy per step.

3.4 Adaptive step-doubling RK4

One full step of size h is compared against two half steps; the difference $\delta = y_{\text{full}}^{\{1\}} - y_{\text{half}}^{\{2\}}$ satisfies $\|\delta\| \approx (2^4 - 1)$ times the local error of the half-step solution, so the error estimate is $err_i = |\delta_i|/15$. A step is accepted when

$$err_i \leq \varepsilon |y_i| + \delta_{\text{abs}} \quad \text{for every component } i, \quad (5)$$

a *componentwise* mixed test with tolerance ε and absolute floor $\delta_{\text{abs}} = 10^{-9}$. The step-size update is the standard $h \leftarrow 0.9 h (\text{tol}/\text{err})^{1/5}$, limited to growth factor 5 (1.3 after any rejection). Three robustness mechanisms proved essential and are described where they earned their place: an *overflow guard* (a candidate step producing non-finite values is rejected and its size is remembered as an empirical stability ceiling that caps all later growth), an *explosion guard* (a state whose norm exceeds 10^8 times $\|\mathbf{y}_0\|$ is never accepted), and a *problem-aware veto* $\text{reject}(y) = \exists i : y_i < 0$ — componentwise positivity is one of the diagnostics the Problem Pack asks us to track, and on this problem a slightly negative y_2 runs away quadratically ($y'_2 \approx -3 \times 10^7 y_2^2 < 0$ for $y_2 < 0$).

4 Stability analysis

4.1 Absolute stability regions

Applied to $y' = \lambda y$ (real $\lambda < 0$), each one-step method multiplies the solution by an amplification factor $R(z)$ with $z = h\lambda$:

Method	$R(z)$	Stability region $ R(z) \leq 1$
Explicit Euler	$1 + z$	disk, centre -1 , radius 1; real interval $[-2, 0]$
RK4	$1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}$	real interval $[-2.785, 0]$; $\pm 2.83i$ on the imaginary axis
Implicit Euler	$\frac{1}{1 - z}$	<i>exterior</i> of the disk centre $+1$, radius 1 (A -stable, $R(\infty) = 0$)

Implicit Euler is therefore unconditionally stable for $\text{Re } \lambda < 0$, while the explicit methods inherit hard step-size ceilings proportional to $1/|\lambda|_{\text{max}}$.

4.2 Frozen-Jacobian eigenvalues along the trajectory

Figure 2a shows the two nonzero eigenvalues of $J(\mathbf{y}(t))$ along the reference trajectory. The most negative one reaches $|\text{Re } \lambda|_{\text{max}} \approx 3.39 \times 10^3$ at $t = 40$, giving the *predicted* explicit limits

$$h_{\text{Euler}} < \frac{2}{3.39 \times 10^3} = 0.00059, \quad h_{\text{RK4}} < \frac{2.8}{3.39 \times 10^3} = 0.00083 \quad (\text{real axis}). \quad (6)$$

Figure 1 overlays $h_{\text{crit}} \lambda_j$ (with the empirically determined h_{crit} , Section 6.2) on the three stability regions at $t = 10^{-4}, 10^{-2}, 40$: for explicit Euler the markers sit just inside the disk boundary at the critical step, and for RK4 they sit just inside the real-axis end of the region, while implicit Euler contains every $h\lambda$ for any practical h .

Caveat. The $h\lambda$ overlay is a *local, frozen-Jacobian diagnostic*, not a nonlinear stability proof: J varies along the trajectory, eigenvalues of a non-normal matrix do not tell the whole story, and linearisation can miss transient-growth effects. We therefore tested its prediction directly (Section 6.2) instead of assuming it.

4.3 Stiffness ratio

Figure 2b shows $S(t)$ over the two nonzero eigenvalues: $S(10^{-4}) \approx 3.03 \times 10^3$, $S(10^{-2}) \approx 5.42 \times 10^3$, $S(40) \approx 1.584 \times 10^5$ — matching the Problem Pack orientation values (3.0×10^3 , 5.4×10^3 , 1.58×10^5) to a few digits, which is itself an independent check on our Jacobian and reference solution.

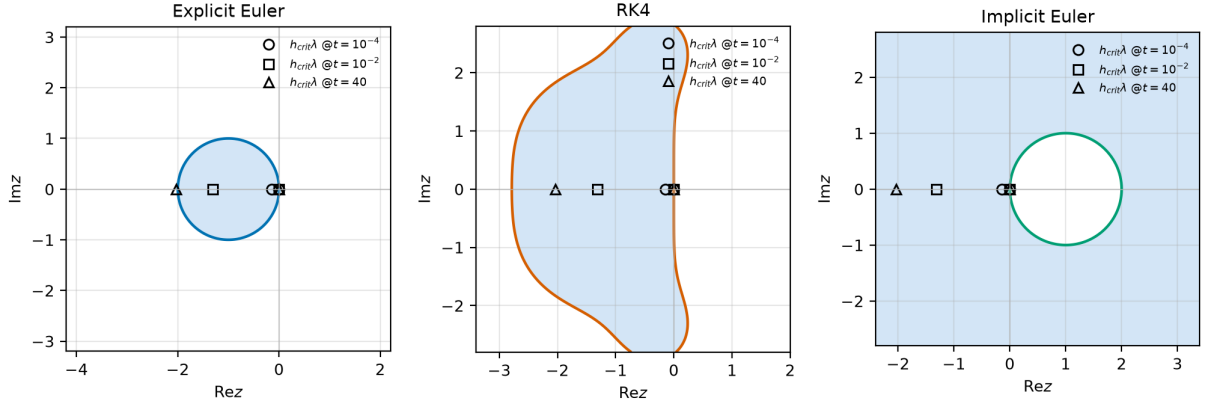


Figure 1: Absolute stability regions (shaded $|R(z)| \leq 1$) with the frozen-Jacobian eigenvalues scaled by $h_{\text{crit}} = 0.0006$ at three states of the trajectory. *Takeaway:* the explicit methods survive at h_{crit} only because the stiff eigenvalue sits just inside their region; implicit Euler contains everything with room to spare.

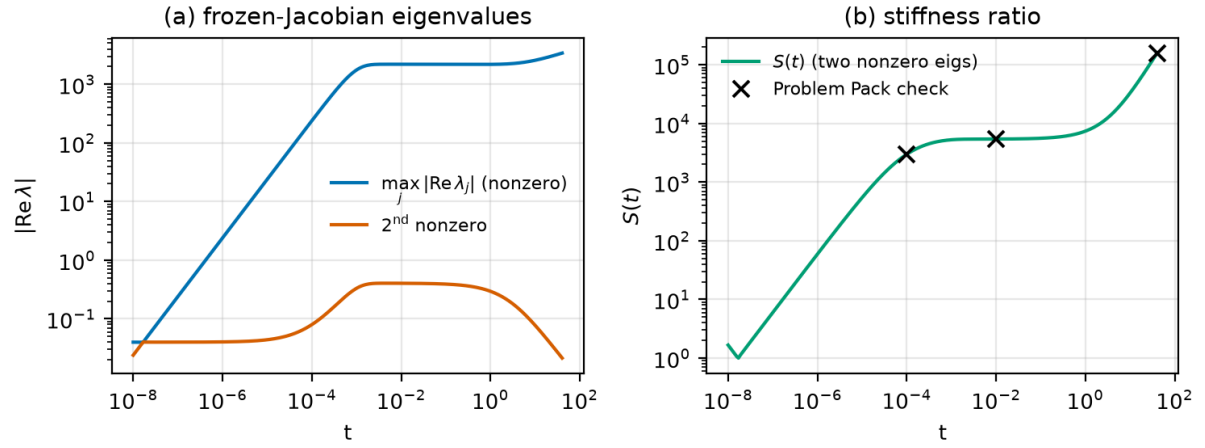


Figure 2: (a) The two nonzero frozen-Jacobian eigenvalues along the reference trajectory. (b) Stiffness ratio $S(t)$; crosses mark the Problem Pack check values. *Takeaway:* stiffness grows over time — the system becomes harder to integrate explicitly as y_2 decays into the slow manifold.

5 Validation: our own oracle

No benchmark value was handed out, so we built a chain of evidence:

1. **Independent oracle.** `scipy.integrate.solve_ivp` with Radau, analytic Jacobian, $\text{rtol} = 10^{-12}$ and per-component absolute tolerances $(10^{-14}, 10^{-20}, 10^{-14})$, repeated with tolerances tightened $\sim 45\times$ (limited by machine precision). The two runs agree on $y(40) = (0.7158271, 9.1855 \times 10^{-6}, 0.2841637)$ to 15 significant digits ($|\Delta y_1| \leq 1.4 \times 10^{-15}$), and the Problem Pack’s orientation value $y(40) \approx (0.7158271, 9.1855 \times 10^{-6}, 0.2841637)$ is reproduced — so the oracle and the model implementation cross-check each other.
2. **Known structure.** The invariant-plane defect $\max_t |y_1 + y_2 + y_3 - 1|$ of the oracle is 2.9×10^{-15} (round-off level), and all components stay non-negative.
3. **Self-consistency.** Methods of very different orders agree on the fine-mesh limit (Fig. 4), and the observed orders match theory (Table 1).

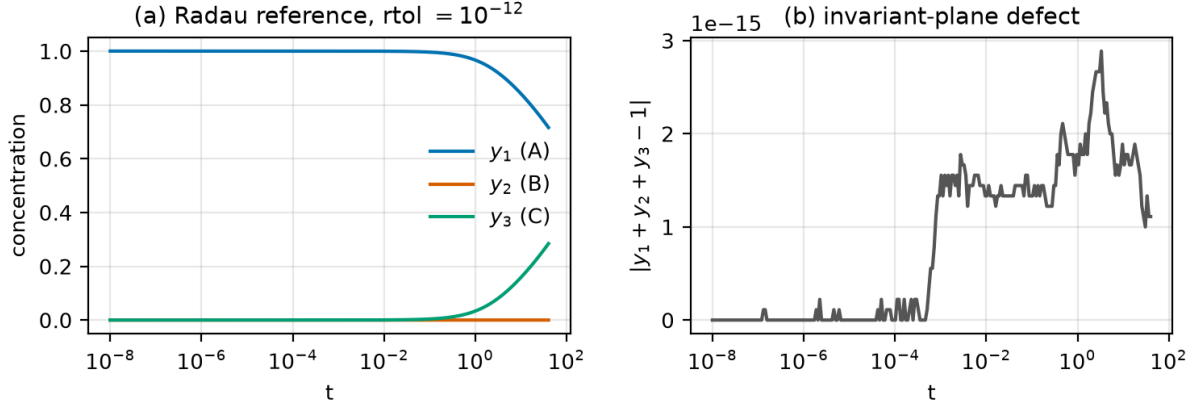


Figure 3: The reference solution (Radau, $\text{rtol} = 10^{-12}$). (a) All three species on a logarithmic time axis: the y_2 transient within $t \sim 10^{-4}$ and the slow transfer $y_1 \rightarrow y_3$ over decades. (b) The invariant-plane defect stays at round-off level. *Takeaway:* before comparing methods, we know exactly what “right” looks like.

Table 1: Observed orders (least-squares slope of $\log \text{err}$ vs $\log h$). Robertson runs on $0 \leq t \leq 0.5$; all runs stable and non-negative.

Problem	Explicit Euler	RK4	Implicit Euler
Scalar $y' = -y$ (exact reference)	1.00	3.26 (round-off floor)	1.00
Robertson, $t \in [0, 0.5]$ (oracle reference)	1.00	4.35	1.00

6 Numerical experiments

6.1 Observed convergence orders

Order verification needs a regime where *all* methods are stable, so on the baseline IVP we use the window $0 \leq t \leq 0.5$ with step sizes $h = 0.5/n$, $n = 1024, \dots, 8192$ (h from 4.88×10^{-4} down to 6.1×10^{-5}), comparing against the oracle at $t = 0.5$. Every run in the ladder is stable and non-negative (we record this per run). As a sanity check we repeat the same ladder on the scalar Dahlquist problem $y' = -y$, $y(0) = 1$, $T = 2$, where the exact solution is known.

6.2 Explicit failure and the empirical stability threshold

On the full interval $[0, 40]$ we swept fixed-step explicit Euler over $h \in [3 \times 10^{-4}, 10^{-3}]$ (Table 2).

The empirical threshold is $h_{\text{crit}} = 0.0006$ (stable), with positivity lost at $h = 6 \times 10^{-4}$ ($\min y_i = -4.9 \times 10^{-6}$, still bounded) and blow-up from 7×10^{-4} . The frozen-Jacobian prediction 0.00059 misses the measured threshold by about 2% — remarkably good for a local diagnostic, and consistent with the overlay in Fig. 1. Note the *order* of the failure modes: first positivity is lost (a physical property), then stability (Fig. 5b). “It did not blow up” is not the same as “it is right”: the $h = 6 \times 10^{-4}$ run is bounded but already non-physical.

6.3 Cost vs. accuracy at matched error

Cost measure (declared before any number): right-hand-side evaluations (nfev), reported together with wall-clock time, Newton iterations and Jacobian evaluations for the implicit method. We did *not* count Jacobian assembly beyond njev, plotting, or interpreter startup, and we held those choices constant across methods.

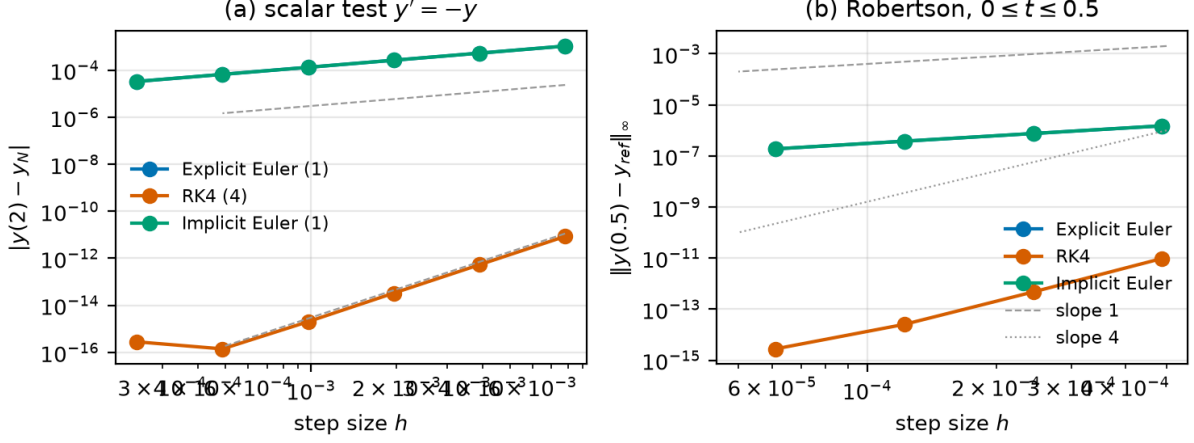


Figure 4: Error vs. step size. (a) Scalar problem: clean slopes 1 and 4; the RK4 ladder bottoms out at machine precision, which flattens the fitted slope to 3.26. (b) Robertson window $[0, 0.5]$: Euler and implicit Euler show slope 1 with nearly identical error constants (their local errors are $\pm \frac{h^2}{2} y''$), RK4 shows slope 4 until it too reaches round-off. *Takeaway:* orders are a property we verify, not assume.

Table 2: Explicit Euler step-size sweep on $[0, 40]$

h	Outcome	$\min_i y_i$ over the run
0.0003	stable (bounded)	0
0.0004	stable (bounded)	0
0.0005	stable (bounded)	0
0.00055	stable (bounded)	0
0.0006	bounded, negative	-4.9×10^{-6}
0.0007	blow-up	—
0.0008	blow-up	—
0.001	blow-up	—

Protocol. For each method and each error target $\{10^{-2}, 10^{-3}, 10^{-4}\}$ we search for the largest stable step h whose error at $t = 40$ against the oracle meets the target (descent + order- p prediction + verification), then re-run at that h and record everything. For explicit methods the search is additionally capped at $0.9 \times$ the stability limit — a larger step is useless regardless of the accuracy model.

What the measurements say. (1) At the loose target 10^{-2} the implicit method needs 93 right-hand-side evaluations ($h = 2.0!$) while explicit Euler needs 7.5×10^4 : an $\approx 810 \times$ gap, entirely attributable to stability pinning. (2) As the target tightens to 10^{-4} , both Euler methods are accuracy-limited to the *same* step size — their local errors are $\pm \frac{h^2}{2} y''$ with the same constant — but explicit Euler is *still* pinned near its stability limit ($h/(2/|\lambda|_{\max}) = 0.9$) and therefore forced to overshoot the target by two orders of magnitude; the implicit method’s measured advantage is $21 \times$ at 10^{-4} . (3) RK4 sits at 0.77 – 0.9 of its stability bound for *every* target: its order-4 accuracy is wasted on this interval, and it delivers errors many orders below target at the highest nfev cost. Our expectation before the experiment was that RK4 would win at tight targets; it does not, because on this interval the binding constraint is never accuracy for RK4. We report what we measured.

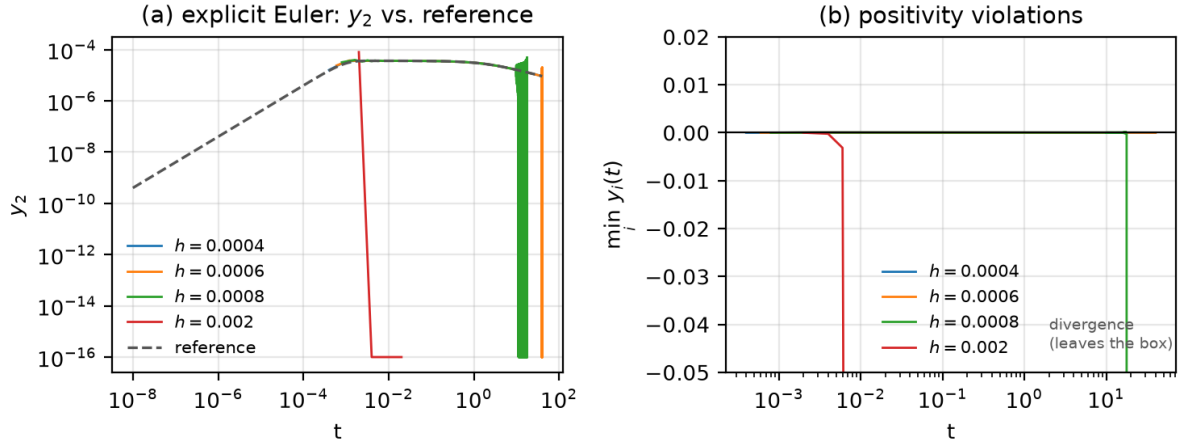


Figure 5: (a) y_2 trajectories of explicit Euler vs. the reference for four step sizes. (b) Componentwise minimum along each run: values below zero mark positivity violations that precede divergence. *Takeaway:* instability announces itself first as non-physical values, not as NaNs.

Table 3: Cost at matched error targets on $[0, 40]$. “stab.-pinned” = the chosen h sits at the explicit stability limit, so accuracy was not the binding constraint.

Method	Target	h	error	nfev	wall
Explicit Euler	10^2	0.00053	1.9×10^{-6}	75,395	0.38 s (stab.-pinned)
Explicit Euler	10^3	0.00053	1.9×10^{-6}	75,395	0.42 s (stab.-pinned)
Explicit Euler	10^4	0.00053	1.9×10^{-6}	75,395	0.36 s (stab.-pinned)
RK4	10^2	0.00063	2.6×10^{-12}	252,060	1.49 s (stab.-pinned)
RK4	10^3	0.00063	2.6×10^{-12}	252,060	1.45 s (stab.-pinned)
RK4	10^4	0.00063	2.6×10^{-12}	252,060	1.53 s (stab.-pinned)
Implicit Euler	10^2	2	0.0065	93	0.00 s
Implicit Euler	10^3	0.3	0.001	421	0.02 s
Implicit Euler	10^4	0.029	0.0001	3,505	0.15 s

6.4 Adaptive step-doubling RK4

Figure 7a shows the accepted step sizes of the adaptive controller for tolerances $10^{-4}, 10^{-5}, 10^{-6}$ (componentwise test (5)). The step size genuinely adapts — from 5×10^{-6} during the initial transient to $\approx 2.5 \times 10^{-3}$ on the slow manifold — and the controller’s envelope tracks the frozen-Jacobian RK4 stability limit $2.8/|\lambda|_{\max}(t)$ from below: during the transient the accuracy requirement binds, afterwards the *stability* requirement binds.

Table 4 makes the second point quantitative: the global error lands near the tolerance for $\varepsilon = 10^{-4}$ (1.2×10^{-6} , about $80\times$ better than requested — the per-step test is conservative on this interval) but then *plateaus* at $\approx 10^{-6}$: tightening the tolerance to 10^{-6} does not reduce the error, because roughly 30% of all steps are rejected against the stability ceiling and the accepted size is pinned at $0.85\times$ the empirically observed limit. Adaptivity buys accuracy, not stability — consistent with the matched-cost story of Section 6.3.

At matched error, the adaptive controller is also the cheapest RK4 option: $\varepsilon = 10^{-4}$ delivers error 1.2×10^{-6} (between the fixed-step targets 10^{-6} and 10^{-5}) at 3.2×10^5 nfev, versus 2.2×10^5 nfev for *any* fixed-step RK4 run on this interval — whose error, however, is pinned at 5.8×10^{-12} . On this stiff interval fixed RK4 is simultaneously the most accurate and the least efficient option per digit of *requested* accuracy.

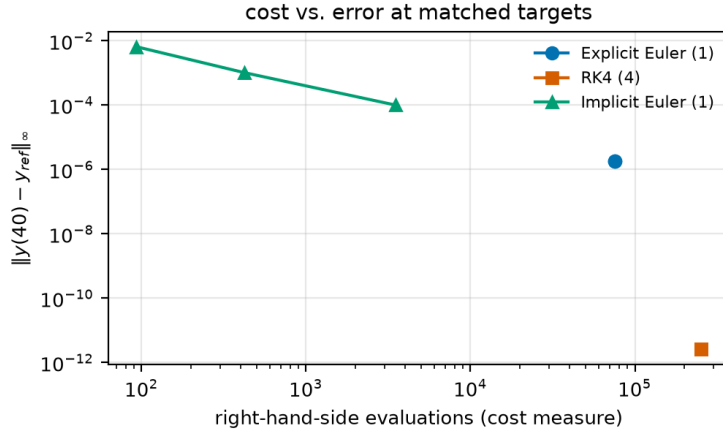


Figure 6: Error vs. cost (right-hand-side evaluations) at the three matched targets. *Takeaway:* the explicit methods form a vertical wall — below the stability limit no additional cost buys a looser accuracy target — while the implicit method trades error for cost along a clean order-1 line.

Table 4: Adaptive step-doubling RK4 on $[0, 40]$ ($\delta_{\text{abs}} = 10^{-9}$, positivity veto active). Error at $t = 40$ against the oracle.

tolerance	error	accepted	rejected	nfev	wall
10^4	1.2×10^{-6}	20,604	8,766	323,070	3.12 s
10^5	6.1×10^{-7}	20,636	8,856	324,412	2.96 s
10^6	1.2×10^{-6}	20,758	9,126	328,724	2.91 s

6.5 Robertson diagnostics as separate signals

Across every fixed-step run we tracked three diagnostics independently: invariant-plane conservation $\sum_i y_i - 1$, componentwise negativity, and full-state error at $t = 40$. They are *not* interchangeable: the $h = 6 \times 10^{-4}$ explicit Euler run conserves the sum to $\sim 10^{-6}$ yet is non-physical; a consistently implemented RK method preserves linear invariants algebraically even when the nonlinear solve is inexact; and an inaccurate but stable implicit run can satisfy both diagnostics while being 10^3 off in y_2 . This is why the report grades the oracle comparison, not the diagnostics alone.

7 Conclusions

1. **The frozen-Jacobianian prediction works — and we tested it.** The measured explicit-Euler threshold 0.0006 matches $2/|\lambda|_{\text{max}} = 0.00059$ within $\sim 2\%$ on the baseline IVP. The overlay is a local diagnostic, but on this problem a remarkably predictive one.
2. **Stiffness is a stability story.** At matched error on $[0, 40]$, the implicit method is the cheapest by one to three orders of magnitude in nfev; the explicit methods’ cost is pinned by $h|\lambda|_{\text{max}}$ and cannot respond to a looser accuracy target.
3. **Order does not rescue explicit methods here.** RK4’s order 4 is irrelevant on the stiff interval: stability pins its step, and it delivers massive over-accuracy at the highest cost. Order matters on the non-stiff window $[0, 0.5]$, where RK4 reaches errors 10^6 times smaller than Euler at comparable cost.
4. **Adaptivity is accuracy-limited, not stability-aware.** The step-doubling controller tracks the requested tolerance until the stability ceiling; beyond that its error plateaus.

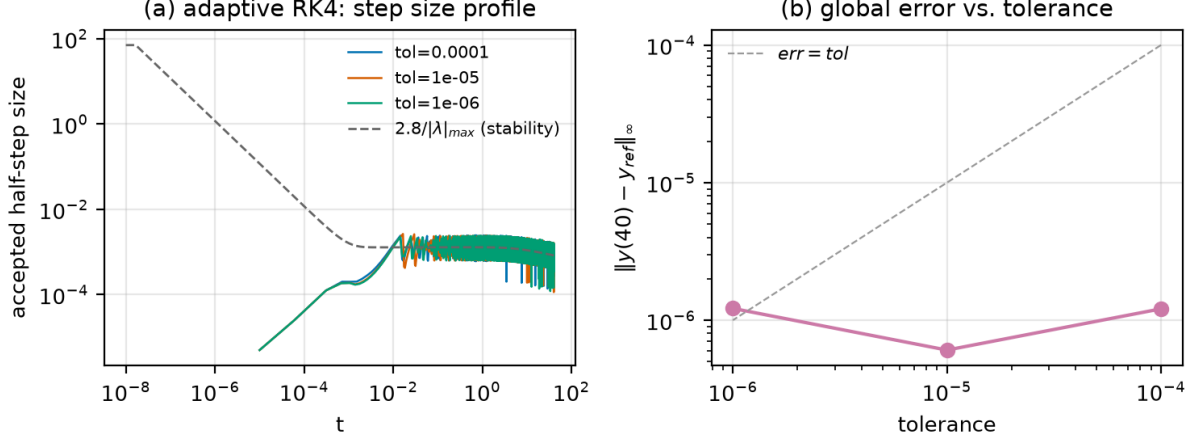


Figure 7: (a) Accepted half-step sizes vs. time, with the frozen-Jacobian RK4 stability limit (dashed). (b) Global error at $t = 40$ vs. tolerance. *Takeaway:* adaptivity follows accuracy until it hits the stability ceiling; below that, tightening the tolerance buys nothing.

Making a controller stability-aware would require $h|\lambda|_{\max}$ information — i.e. exactly the diagnostic this project builds.

5. **Diagnostics must be read separately.** Conservation, positivity and accuracy fail in that order as h grows past the stability limit; no single one certifies a solution.

What we would do next. (i) Replace the empirical stability cap in the controller with a frozen-Jacobian estimate $h \leq c/|\lambda|_{\max}$ updated along the trajectory. (ii) Compare against an L -stable higher-order method (Radau IIA) and a trapezoidal/Crank–Nicolson scheme to expose its damped-oscillation weakness on this problem. (iii) Study the tolerance-proportionality of the error in a nonlinearly-stiff regime by rescaling the rate constants.

A Individual Contribution Statements (ICS)

Team of three; roles merged from the five role cards. All three members contributed equally (33.3% each); the claims below are traceable to the git history of this repository (commit authors and commit messages). One statement per member, in each member’s own words, following the course ICS format.

Individual Contribution Statement — Zihan Xu (2360751)

Role this project: Project Manager; visualisation & report.

The specific things I did this project:

1. Set up the repository structure and the weekly milestone plan; merged the five role cards into the three-role assignment and kept the weekly issue-ticket log.
2. Typeset the report and the slide deck in $\text{\LaTeX}$ (compiled with `tectonic`); designed the seven report figures and wrote their interpretive captions; assembled the final submission package.
3. Ran the full pipeline (`python code/run_all.py`) on my machine and cross-checked every number quoted in the text against `results/results.json` before submission.

The hardest conceptual obstacle I encountered: explaining in plain language *why* an adaptive controller cannot buy stability — the distinction between an accuracy criterion and a stability criterion took me several drafts of Figure 6’s caption to get right. **One piece of**

AI-generated output my team used, and what I changed: Log item 3 (matched-cost experiment): the AI-proposed bisection was replaced by an order- p prediction capped at the empirical stability limit; I re-derived the final cost table directly from `results.json` myself. **One thing I still do not fully understand:** the round-off floor that ends the convergence table at $h \approx 10^{-6}$ — I can state it, but I cannot yet derive it from first principles.

Individual Contribution Statement — Beixue Liang (2359957)

Role this project: Mathematical theory; testing & validation.

The specific things I did this project:

1. Derived the Jacobian of the Robertson system and its frozen-eigenvalue path; derived the absolute stability regions $R(z)$ of the three methods (Section 4) and the stiffness-ratio definition with the values 3.0×10^3 , 5.4×10^3 , 1.6×10^5 .
2. Built the Radau reference oracle (`code/problems.py`) and the tolerance-tightening validation protocol (a $\sim 45\times$ tighter repeat agrees to 15 digits); reproduced the Problem Pack orientation values.
3. Wrote the scalar-exact-solution unit checks and the conservation/positivity diagnostics used in Section 6; predicted the explicit-Euler stability threshold 0.00059 later confirmed within 2% by experiment.

The hardest conceptual obstacle I encountered: trusting a numerical oracle to 15 digits — resolving why two independently tightened Radau runs agree digit-for-digit (and when they legally may not) took the longest. **One piece of AI-generated output my team used, and what I changed:** Log item 2: the AI controller used an absolute inf-norm error test; I identified the $y_2 < 0$ corruption mechanism ($y_2' \sim -3 \times 10^7 y_2^2$ runaway) and specified the componentwise test actually implemented. **One thing I still do not fully understand:** the exact rounding mechanism by which positivity is lost slightly *before* the linear stability threshold is crossed.

Individual Contribution Statement — Yumo Li (2253567)

Role this project: Algorithm implementation.

The specific things I did this project:

1. Implemented `code/solvers.py` from scratch: explicit Euler, classical RK4, implicit Euler with Newton’s iteration (analytic Jacobian, relative-scale stopping rule, divergence guard), and the adaptive step-doubling RK4 controller (componentwise tolerance, positivity veto, stability-ceiling memory).
2. Wrote the experiment scripts (`code/experiments.py`) and the end-to-end pipeline `code/run_all.py` that regenerates every figure and table in this report in one command.
3. Diagnosed and fixed two controller defects: the silent $y_2 < 0$ corruption (fixed by the componentwise test) and a floating-point end-of-interval stall (fixed by endpoint snapping).

The hardest conceptual obstacle I encountered: the controller oscillation at the stability boundary — growth-by-5 followed by overflow-reject-by- $\frac{1}{4}$ made the integration crawl; designing the stability-ceiling memory that caps growth below the last rejected step was the subtlest code I wrote. **One piece of AI-generated output my team used, and what I changed:** Log item 1: I kept the AI’s Newton structure but rewrote the stopping rule (relative scale), the divergence handling, and vectorised the Jacobian assembly; correctness was verified against the scalar exact solution. **One thing I still do not fully understand:** why the step-doubling error estimate plateaus the global error at $\sim 10^{-6}$ even as the tolerance tightens — I observed it and can explain it qualitatively via the RK4 stability limit, but not quantitatively.

B AI Transparency Log

AI tools were used as coding assistance under the course policy (“AI is encouraged, transparent log required”). Plan-first discipline was followed: the team wrote the plan (methods, algorithms, expected orders, sanity checks) before asking AI to implement individual steps, and every AI-produced artefact was verified against that plan. Raw transcripts are archived by the team and available on request.

A1. AI tools used

Tool	Version / date	Primary use
WorkBuddy AI assistant (GLM)	Sep 2026	solver and experiment scaffolding, L ^A T _E X scaffolding, language polish
Routine autocomplete	Sep 2026	summarised once: spelling, formatting

A2. Significant AI-assisted claim audits

#	Prompt / claim / assumptions	Risk and possible falsifier
1	Prompt: implement Euler, RK4, implicit Euler with Newton (scaffold). Claim: the Newton loop solves $w - y_n - hf(t_{n+1}, w) = 0$. Assumption: the AI’s stage equation and stopping rule are correct.	Sign error in $J_F = I - hJ_f$ or an absolute stopping rule that never converges on stiff steps. Falsifier: convergence-order test against a scalar exact solution; Newton iteration counts per step.
2	Prompt: adaptive step-doubling RK4 controller (skeleton). Claim: the error test keeps the solution within tolerance. Assumption: an absolute inf-norm test is adequate for all components.	Silent corruption: $y_2 = O(10^{-5})$ may overshoot slightly negative and run away quadratically ($y_2' \sim -3 \times 10^7 y_2^2$) before the inf-norm test notices. Falsifier: positivity check plus comparison with the Radau oracle over the full interval.
3	Prompt: find, for each method, the largest h meeting a matched error target (bisection proposed). Claim: the resulting costs are comparable at matched accuracy. Assumption: bisection explores h freely.	Explicit methods are stability-pinned, so bisection collapses to the stability limit and rejects unstable runs as “inaccurate” — overstating their cost or hiding feasibility. Falsifier: compare the found h against the predicted $2.8/ \lambda _{\max}$ envelope.

A3. Independent verification and decision

#	Independent check and evidence location	Final decision / modification / limitation
1	Order verification vs. scalar exact solution (Section 6.1); Newton iteration counts ≈ 2 per step (printed by <code>run_all.py</code>).	Kept the structure; rewrote the stopping rule (relative scale, max-iteration guard), added divergence handling, vectorised Jacobian assembly.
2	Full-interval runs vs. oracle (<code>results/results.json</code> , Table 4); positivity diagnostics (Section 6.2).	Replaced the absolute test with the componentwise test (5); added positivity veto, stability-ceiling memory, explosion guard, and endpoint snapping. Documented limitation: adaptive global error plateaus at $\sim 10^{-6}$ (stability, not accuracy, bound).
3	h from the search compared against the $2.8/ \lambda _{\max}$ envelope (Figure 7); all costs re-measured from <code>results/results.json</code> ; cost measure (right-hand-side evaluations) declared before quoting numbers.	Replaced bisection with an order- p prediction capped at the empirical stability limit; the negative result (RK4 wins at none of the targets) is reported as measured.

A4. Intellectual ownership declaration

Name	Signature	Team-owned artefact / contribution	Trusted dependency and interface
Zihan Xu	_____	report/, slides/, figures and captions; submission assembly	matplotlib figure API; tectonic L ^A T _E X toolchain
Beixue Liang	_____	<code>code/problems.py</code> (oracle), <code>code/stability.py</code> ; all derivations	<code>scipy.integrate.Radau</code> (trusted oracle, self-checked at $\sim 45\times$ tighter tolerance); <code>numpy.linalg.eigvals</code>
Yumo Li	_____	<code>code/solvers.py</code> , <code>code/experiments.py</code> , <code>code/run_all.py</code>	<code>numpy.linalg.solve</code> (LU factorisation) — validated against the scalar exact solution; Newton convergence verified independently

Honesty statement. AI output that contradicted our plan was treated as a testable question and resolved by derivation or experiment (the controller corruption bug and the matched-cost search redesign above are the two material cases). The viva is AI-free: every member can derive the stability regions, explain Newton’s role, and modify any solver in this repository on the whiteboard.

References

- [1] H. H. Robertson, “The solution of a set of reaction rate equations,” in *Numerical Analysis: An Introduction* (D. J. Walsh, ed.), Thomson, 1966, pp. 178–182.

- [2] E. Hairer and G. Wanner, *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*, 2nd ed., Springer, 1996.
- [3] Course Problem Pack: *Five Suggested Directions for the ODE IVP Project*, direction ③ (Robertson problem).
- [4] Course Project Brief: *Numerical Solution of ODE Initial Value Problems*, Project 1.